Ursinus
COLLEGE

CS375 Spring 2026

Ralph Rostock

Week 2

Version Control, Software Development Methodologies, Software Project Brainstorming & Team Formations

How Software Is Built in Industry Today

- Product teams
  - Cross-functional (engineering, product, design, QA)
  - Long-lived teams owning a product, not short projects
  - Teams responsible for success after release
- Continuous delivery
  - Software is always in a “potentially releasable” state
  - Small, frequent changes instead of large releases
  - Automation enables speed and safety

Why might companies prefer frequent, smaller releases over fewer, larger ones?

Traditional Development Models

- Waterfall overview
  - Linear sequence of phases
  - Heavy upfront requirements and design
  - Testing late in the process
- Strengths
  - Predictable milestones
  - Works well when requirements are stable
  - Clear documentation and contracts
- Weaknesses
  - Late feedback
  - High cost of change
  - Risk of building the wrong system

Agile Manifesto

Core Values

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

https://agilemanifesto.org/principles.html

Agile Manifesto (Continued)

- Why the Manifesto was created
  - Reaction to heavyweight, document-driven processes
  - Focus on uncertainty and change
- Core Values
  - “Over” does not mean “instead of”
  - Balance, not elimination

Scrum Overview

- Roles
  - Product Owner: Values and Priorities
  - Scrum Master: Process and Impediments
  - Development Team: Builds the Product
- Ceremonies
  - Sprint Planning
  - Daily Standups
  - Sprint Review
  - Retrospective
- Artifacts
  - Product Backlog
  - Sprint Backlog
  - Increment

Agile in Practice

- What teams really do vs. Textbook Scrum
  - Hybrid processes
  - Fewer ceremonies, more communication
  - Adjust practices over time
- Alternatives to Scrum
  - Kanban: flow-based, no fixed sprints
  - Extreme Programming: engineering discipline (TDD, pair programming)
  - Lean: minimize waste, optimize the whole system

Why Version Control Matters

- Enables collaboration without overwriting work
- Allows multiple people to work on the same codebase safely
- Maintains a complete history of changes
- Records who changed what, when, why
- Makes experimentation low-risk
- Supports rollback and recovery from mistakes

Version control is essential infrastructure for team-based software development.

Core Git Concepts

- Repository
  - Shared project workspace
  - Contains source code, configuration, documentation – anything related to project that you want to track
  - Exists locally and remotely (e.g. GitHub)
- Commit
  - Snapshot of a logical change
  - Includes author, timestamp, and message
  - Serves as technical documentation

Core Git Concepts (continued)

- Branch
  - Independent line of development
  - Enables parallel work
  - Keeps unfinished work isolated
  - Can be permanent or temporary
- Merge
  - Combines changes from different branches
  - Integrates completed work into shared codebase
  - Conflicts are expected and manageable

Typical Team Git Workflow

- Main branch represents stable, working software
- New work happens on feature branches
- Developers commit small, focused changes
- Pull requests propose changes for integration
- Automatic checks run before merging
- Code review provides feedback and quality control, often a requirement for approving a pull request

Git workflows create structured feedback loops

Common Git Mistakes

- Working directly on the main branch
- Making large, unfocused commits
- Skipping code reviews
- Writing poor or vague commit messages
- Treating Git as a backup tool instead of a collaboration tool

Most Git problems result from poor process, not Git itself

How Git Supports Agile Software Development

- Enables Incremental Development
- Supports frequent integration and delivery
- Allows multiple features to be developed in parallel
- Provides traceability from work items to code changes
- Makes feedback and iteration visible

Agile practices depend on version control to scale